
FLASHREG: GPU-ACCELERATED 3-CLIQUE POINT CLOUD REGISTRATION FOR REAL-TIME CORRESPONDENCE-TO-POSE ESTIMATION

Ziyang Yu

School of Computing
Institute of Science Tokyo
Tokyo

yu.ziyang.2001@gmail.com

Xiang Li

School of Computing
Institute of Science Tokyo
Tokyo

652022230010@smail.nju.edu.cn

Qiong Chang

School of Computing
Institute of Science Tokyo
Tokyo

q.chang@c.titech.ac.jp

Jun Miyazaki

School of Computing
Institute of Science Tokyo
Tokyo

miyazaki@comp.isct.ac.jp

ABSTRACT

Graph-based point cloud registration achieves high robustness by identifying geometrically consistent correspondence sets, but constructing second-order compatibility graphs and enumerating candidate cliques remain compute- and memory-intensive. This work presents FlashReg, a GPU-oriented correspondence-to-pose estimator that avoids materializing the dense scored second-order graph. Its Fast First- and Second-Order Graph (FFSOG) construction builds a capacity-bounded sparse second-order graph directly from the binary first-order graph. A dataflow-optimized three-node clique (3-clique) search then selects pivots from compact per-row candidate pools and enumerates triples through sorted sparse-neighborhood intersections. Across indoor and outdoor benchmarks, FlashReg reduces correspondence-to-pose latency by $2\text{--}3\times$ relative to TurboReg at comparable registration recall, while using about 50% of its peak allocated tensor memory on an embedded GPU. These results make FlashReg suitable as a high-throughput registration backend within onboard perception pipelines.

Keywords Point Cloud Registration · Parallel Computing · Graph Processing

1 Introduction

Point cloud registration recovers the rigid transformation that aligns two independently captured scans into a common coordinate frame, which underpins robot perception, SLAM, autonomous driving, and virtual reality [1, 2, 3]. Compared with 2D images, point clouds are sparse and carry no canonical orientation, so they expose far fewer distinctive features and far lower cross-scan similarity; registration has therefore long been burdened by high computational cost and memory requirements. Most real deployments, however, demand real-time response and high registration quality at the same time, especially on the resource-limited platforms where these algorithms actually deploy.

Graph-based registration represents each putative correspondence as a node and connects pairs that satisfy pose-invariant geometric consistency [4, 5, 6]. Cliques in this graph isolate self-consistent correspondence sets, while second-order compatibility further suppresses spurious edges, enabling strong robustness under high outlier rates [6, 7, 8]. This accuracy is expensive: graph construction grows quadratically with the correspondence count, and maximal-clique search may enumerate millions of overlapping candidates. TurboReg [9] replaces exhaustive enumeration with GPU-based pivot search, extending each high-scoring second-order edge into a bounded set of 3-cliques and achieving

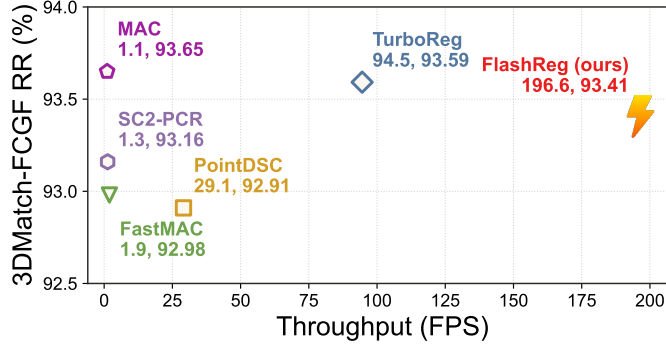


Figure 1: Speed-accuracy trade-off on 3DMatch with FCGF descriptors; each label gives estimator throughput (FPS) and recall (%), and the truncated recall axis magnifies sub-point differences. FlashReg stays within half a point of the strongest evaluated solvers at nearly 200 FPS.

more than a $100\times$ speedup over CPU-based MAC. However, it still materializes dense compatibility matrices and rebuilds intermediate results between stages. The resulting computation and memory traffic are particularly costly on unified-memory embedded GPUs.

Building on the 3-clique formulation, we present **FlashReg**, a clique-based registration framework that retains TurboReg’s pivot-based 3-clique scoring objective while using a capacity-bounded candidate set. Our design is guided by two structural observations. First, the graph is sparse: each correspondence is compatible with only a small fraction of the others, making a dense representation unnecessary. Second, graph construction already produces much of the information subsequently needed for pivot selection and clique enumeration. By co-designing graph construction and inter-stage data movement at the kernel level, these intermediate results can be passed directly to the following processes rather than recomputed, substantially reducing the computational and memory overhead of clique enumeration.

These observations motivate two designs in FlashReg: (1) Fast First- and Second-Order Graph (FFSOG) construction, which computes and compacts SOG rows using on-chip buffers; and (2) a dataflow-optimized 3-clique search, which fuses pivot selection with SOG construction and enumerates cliques directly from the compact sparse representation, avoiding the write-back of a dense SOG score matrix. At the cost of a slightly more elaborate construction kernel, FlashReg removes most of the memory traffic and computation of the two most expensive stages of the pipeline. As illustrated in Fig. 1, FlashReg stays within half a recall point of the strongest evaluated solvers at an estimator throughput of nearly **200 FPS**. The savings are larger still on embedded GPUs: FlashReg uses about 50% of TurboReg’s peak allocated tensor memory and consumes less measured GPU-rail energy at comparable registration recall. FlashReg thereby turns accurate 3-clique registration from a dense, stage-separated pipeline into a compact, memory-efficient backend suitable for resource-constrained robot platforms. The main contributions are summarized as follows:

- A compact, GPU-native graph construction method that removes the dense SOG score-matrix bottleneck from 3-clique registration. FFSOG fuses first-order compatibility evaluation and constructs the scored SOG directly in a capacity-bounded sparse layout, avoiding materialization of the dense score matrix while retaining comparable registration recall.
- A dataflow-optimized 3-clique search algorithm that co-designs pivot selection and clique enumeration with graph construction. It selects pivots directly from row-local candidates produced during SOG construction and enumerates cliques through sorted sparse-neighborhood intersections, avoiding dense intermediate storage, redundant computation, and host synchronization.
- Experiments on indoor, outdoor, and low-overlap benchmarks show that FlashReg preserves competitive recall while reducing estimator latency by more than $2\times$; on the Jetson AGX Xavier, it uses about 50% of TurboReg’s peak allocated tensor memory with lower measured GPU-rail energy.

2 Related Work

2.1 Descriptor-based Point Cloud Registration

ICP [10] requires a good initial pose, while globally optimal variants such as Go-ICP [11] remain far from real time. Descriptor-based methods avoid initialization by matching handcrafted [12, 13, 14, 15] or learned [16, 17, 18] local

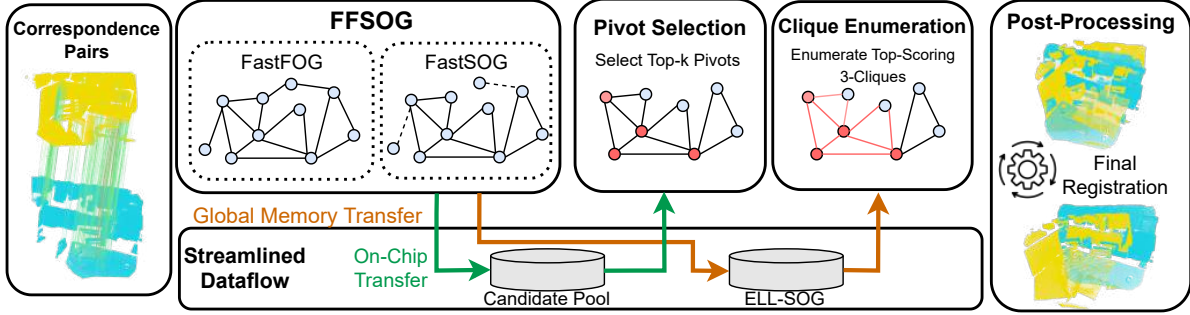


Figure 2: FlashReg pipeline. FastFOG and FastSOG form FFSOG, which emits column-sorted ELL-SOG rows and per-row top- m candidate pools without materializing the dense SOG score matrix. The dataflow-optimized search consumes these outputs in place. Green and orange arrows denote on-chip and global-memory traffic, respectively.

features; Predator [19] and GeoTransformer [20] further add overlap-aware matching. Even so, low-overlap and noisy scans leave many outliers, making robust filtering essential.

2.2 Graph-based Point Cloud Registration

Graph-based registration represents correspondences as nodes and connects pose-invariant compatible pairs [4]; a clique then forms a self-consistent inlier set. Learning-based filters such as PointDSC [21], DGR [22], and 3DRegNet [23] require labeled training, whereas geometric methods search compatibility graphs directly. TEASER++ [5] combines clique pruning with GNC, and SC2-PCR [6] uses second-order compatibility to suppress spurious edges. MAC [7] loosens clique-enumeration constraints and estimates a pose per clique, improving low-overlap accuracy. This robustness introduces two bottlenecks: graph construction evaluates and stores $O(N^2)$ pairwise relations, while maximal-clique enumeration can generate millions of overlapping candidates that must be scored and converted into pose hypotheses.

2.3 Registration Algorithm Acceleration

General acceleration methods optimize CPU solvers [24, 25, 26, 27] or parallel descriptor workloads on GPUs, but recursive clique search and its data dependencies remain difficult to parallelize. FastMAC [28] samples the graph before enumeration to reduce candidates, but aggressive sampling degrades accuracy. TurboReg [9] instead ports the pipeline to the GPU and replaces maximal-clique enumeration with pivot-based 3-clique search. Since three non-collinear correspondences determine a rigid transformation, it achieves over $100\times$ speedup versus CPU-based MAC. However, TurboReg still materializes the dense SOG score matrix, applies dense row operations, and rebuilds intermediate results between stages. FlashReg retains TurboReg’s search objective while addressing these remaining graph-construction and data-movement costs through sparse SOG construction and a coordinated GPU dataflow.

3 Method

Given a source and a target point cloud, descriptor matching yields a set of N putative correspondences $\mathcal{C} = \{c_i\}_{i=1}^N$, where each $c_i = (p_i^s, p_i^t)$ pairs a source point p_i^s with a target point p_i^t . Registration seeks the rigid transformation that aligns the inlier correspondences,

$$(\mathbf{R}^*, \mathbf{t}^*) = \arg \min_{\mathbf{R} \in SO(3), \mathbf{t} \in \mathbb{R}^3} \sum_{i \in \mathcal{I}} \|\mathbf{R} p_i^s + \mathbf{t} - p_i^t\|_2^2, \quad (1)$$

where $SO(3)$ denotes the group of 3D rotations ($\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = 1$), and $\mathcal{I}$ denotes the unknown subset of inlier correspondences. FlashReg searches for geometrically consistent 3-cliques to generate pose hypotheses, which are subsequently verified against all correspondences.

As illustrated in Fig. 2, FlashReg organizes second-order-graph clique registration into four stages: FastFOG, FastSOG, pivot selection, and clique enumeration. FastFOG and FastSOG together form FFSOG (Sec. 3.1), while the dataflow-optimized search (Sec. 3.2) covers the latter two stages. The retained 3-cliques become pose hypotheses, which are verified and refined as described in Sec. 3.3.

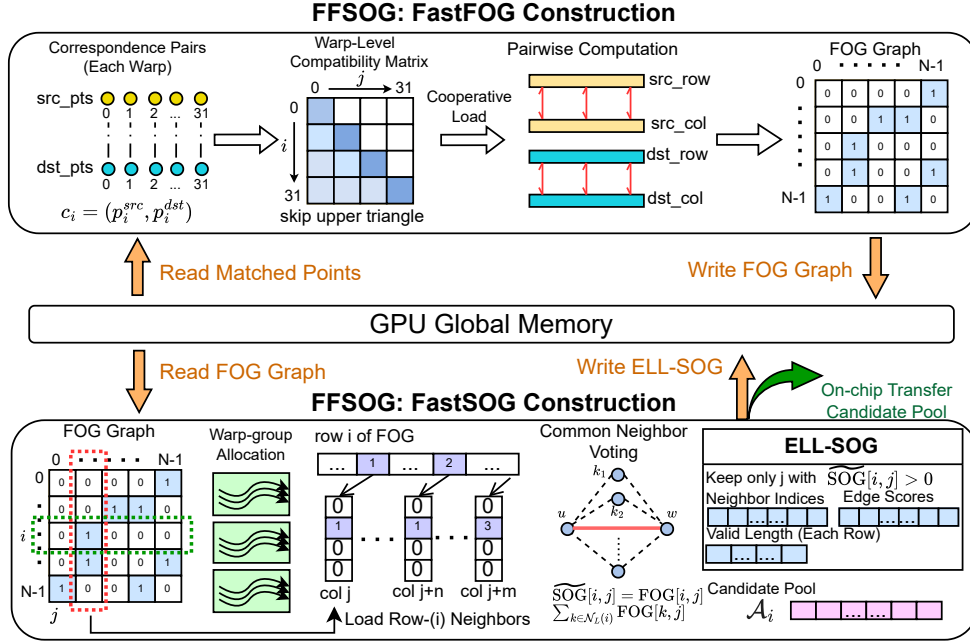


Figure 3: FFSOG. Top, FastFOG: a fused, tiled kernel builds the FOG adjacency matrix and writes it to global memory once. Bottom, FastSOG: one block per node computes a row of the SOG score matrix in shared memory by common-neighbor voting and compacts the nonzeros via warp ballot and prefix sum; before the row leaves shared memory, its m highest-scoring retained edges are extracted into the compact pivot-candidate pool $\mathcal{A}_i$.

3.1 FFSOG: Fast First- and Second-Order Graph Construction

Fig. 3 illustrates the two modules of FFSOG. The upper FastFOG path builds the first-order graph (FOG) with a single fused, tiled kernel and writes its adjacency matrix to global memory once. The lower FastSOG path converts the FOG into the weighted second-order graph (SOG) row by row: one thread block gathers a node’s FOG neighbors into shared memory, computes its SOG edge scores by common-neighbor voting, and emits the positive entries directly as a column-sorted sparse adjacency, the ELL-SOG, alongside a per-row pool of the top-scoring edges for the subsequent pivot selection. FlashReg retains the dense binary FOG adjacency matrix for first-order lookups. The larger dense SOG score matrix, however, is never materialized. The following sections describe each module; rows that exceed the configured capacity fall back to a capped approximation, quantified in Secs. 3.2 and 4.4.

3.1.1 FastFOG Construction

First-order compatibility tests whether a pair of correspondences preserves its intra-cloud distance. To distinguish a graph from its representation, we use FOG and SOG for the graphs and $\mathbf{M}_{\text{FOG}}$ and $\mathbf{M}_{\text{SOG}}$ for their matrices. In particular, let $\mathcal{G}_{\text{FOG}} = (\mathcal{C}, \mathcal{E}_{\text{FOG}})$ denote the undirected FOG and let $\mathbf{M}_{\text{FOG}} \in \{0, 1\}^{N \times N}$ denote its symmetric adjacency matrix.

As shown in the upper path of Fig. 3, FastFOG evaluates the compatibility tests with one fused, tiled kernel. A 32×32 block produces one tile of $\mathbf{M}_{\text{FOG}}$, with each warp comparing one row correspondence against 32 column correspondences. FastFOG evaluates only the lower-triangular half of the matrix and writes every off-diagonal result to both $[\mathbf{M}_{\text{FOG}}]_{ij}$ and $[\mathbf{M}_{\text{FOG}}]_{ji}$. It therefore produces the complete dense binary adjacency matrix; FastSOG later stores only the strict upper-triangular positive SOG entries. Let i_0 and j_0 be the tile origins; for local indices $a, b \in \{0, \dots, 31\}$, the block cooperatively stages the row points $(p_{i_0+a}^s, p_{i_0+a}^t)$ and column points $(p_{j_0+b}^s, p_{j_0+b}^t)$ in shared memory (Fig. 3, top). After synchronization, thread (a, b) reuses these arrays with $i = i_0 + a$ and $j = j_0 + b$, and computes

$$\ell_{ij}^s = \|p_i^s - p_j^s\|_2, \quad \ell_{ij}^t = \|p_i^t - p_j^t\|_2, \quad d_{ij} = |\ell_{ij}^s - \ell_{ij}^t|, \quad (2)$$

followed by the binary compatibility score

$$[\mathbf{M}_{\text{FOG}}]_{ij} = \mathbf{1}(i \neq j) \mathbf{1}(d_{ij} < \tau) \mathbf{1}(\ell_{ij}^s + \ell_{ij}^t > r), \quad (3)$$

where $\mathbb{1}(\cdot)$ is the indicator function, which equals one when its argument is true and zero otherwise. Thus, $(i, j) \in \mathcal{E}_{\text{FOG}}$ exactly when $[\mathbf{M}_{\text{FOG}}]_{ij} = 1$. Both ℓ_{ij}^s and ℓ_{ij}^t are Euclidean distances, so the length-consistency threshold τ and NMS radius r use the same length unit as the point coordinates.

Compared with stage-by-stage construction, FastFOG fuses all compatibility operations into one kernel, reuses tiled coordinates in shared memory, and computes only one triangle of the symmetric matrix. It produces the same FOG adjacency matrix with $O(N^2)$ computation and storage while reducing constant-factor computation and global-memory traffic, cutting construction time from 1.64 to 0.19 ms (Sec. 4.4).

3.1.2 FastSOG Construction

FastSOG is designed around a mismatch between SOG computation and its downstream use. The original method materializes an $N \times N$ scored matrix. General-purpose sparse matrix multiplication incurs metadata construction, symbolic discovery, and output allocation that are unnecessary for our fixed consumer-oriented layout. FastSOG instead exploits the binary masked-voting structure to score and compact each row directly into the two representations consumed downstream.

Let $\mathbf{M}_{\text{SOG}} \in \mathbb{R}_{\geq 0}^{N \times N}$ denote the symmetric score matrix of the weighted SOG. It is computed directly from the binary FOG adjacency matrix by a masked matrix product,

$$\begin{aligned} \mathbf{M}_{\text{SOG}} &= \mathbf{M}_{\text{FOG}} \odot (\mathbf{M}_{\text{FOG}} \times \mathbf{M}_{\text{FOG}}), \\ [\mathbf{M}_{\text{SOG}}]_{ij} &= [\mathbf{M}_{\text{FOG}}]_{ij} \sum_{k=1}^N [\mathbf{M}_{\text{FOG}}]_{ik} [\mathbf{M}_{\text{FOG}}]_{kj}, \end{aligned} \quad (4)$$

where $\odot$ denotes element-wise product and $\times$ denotes matrix product. The matrix-product term counts the common FOG neighbors shared by i and j . To describe this neighborhood structure, let $\mathcal{N}_{\text{FOG}}(i) = \{k : [\mathbf{M}_{\text{FOG}}]_{ik} = 1\}$ denote the FOG neighborhood of node i , with degree $d_i = |\mathcal{N}_{\text{FOG}}(i)|$. In Eq. (4), the outer factor $[\mathbf{M}_{\text{FOG}}]_{ij}$ forces the SOG score to zero whenever correspondences i and j are not connected in the FOG. The SOG therefore never introduces a new edge; it only assigns a common-neighbor score to an existing FOG edge. Consequently, row i of $\mathbf{M}_{\text{SOG}}$ can contain at most d_i nonzero entries, and only neighbors $k \in \mathcal{N}_{\text{FOG}}(i)$ can contribute to their scores. A dense matrix multiplication nevertheless evaluates all N^2 output cells using length- N inner products. With mean FOG degree $\bar{d} \ll N$, at most a fraction $\bar{d}/N$ of these cells can carry useful information.

FastSOG exploits both levels of sparsity in the lower path of Fig. 3. One thread block handles each row i , scans row i of $\mathbf{M}_{\text{FOG}}$ to obtain its candidate columns $\mathcal{N}_{\text{FOG}}(i)$, and retains up to L of them as a shared-memory voter list $\mathcal{N}_L(i) \subseteq \mathcal{N}_{\text{FOG}}(i)$; when $d_i > L$, the list keeps the first L neighbors encountered by the block’s strided row scan, an implementation-defined subset. The expensive common-neighbor vote is evaluated only for $j \in \mathcal{N}_{\text{FOG}}(i)$ with $j > i$:

$$[\widetilde{\mathbf{M}}_{\text{SOG}}]_{ij} = [\mathbf{M}_{\text{FOG}}]_{ij} \sum_{k \in \mathcal{N}_L(i)} [\mathbf{M}_{\text{FOG}}]_{kj}. \quad (5)$$

Here, $\widetilde{\mathbf{M}}_{\text{SOG}}$ denotes FlashReg’s capacity-bounded approximation of the exact score matrix $\mathbf{M}_{\text{SOG}}$. Each retained neighbor k contributes one vote exactly when it also connects to j . The initial row scans cost $O(N^2)$, while voting costs $O(\sum_i d_i \min(d_i, L))$, eliminating the cubic dense matrix-product work; the overall graph pipeline still includes FastFOG’s unavoidable quadratic pairwise evaluation. The scores remain in registers or shared memory and are compacted in increasing column order by a warp ballot and prefix sum. To make the output layout explicit, let $j_{i,1} < \dots < j_{i,z_i}$ be the z_i columns satisfying $j > i$ and $[\widetilde{\mathbf{M}}_{\text{SOG}}]_{ij} > 0$, let $\ell_i = \min(z_i, L)$, and write $s_{i,t} = [\widetilde{\mathbf{M}}_{\text{SOG}}]_{i,j_{i,t}}$. The three rows shown in the ELL-SOG box of Fig. 3 are

$$\begin{aligned} \text{Neighbor Indices}_i &= [j_{i,1}, \dots, j_{i,\ell_i}, \underbrace{*, \dots, *}_{L-\ell_i}], \\ \text{Edge Scores}_i &= [s_{i,1}, \dots, s_{i,\ell_i}, \underbrace{*, \dots, *}_{L-\ell_i}], \\ \text{Valid Length}_i &= \ell_i. \end{aligned} \quad (6)$$

Here, $*$ denotes padding ignored by downstream kernels. The fixed-width padded rows, together with an explicit per-row valid length, form an ELLPACK-R layout [29]; we term the resulting structure the *ELL-SOG*. Its fixed row stride keeps every row directly addressable by index on the GPU, while storing valid entries in ascending *column* order enables the binary-search intersection described in Sec. 3.2. Before a compacted row leaves shared memory, the block additionally extracts its m highest-scoring edges ($m=32$) into a per-row *candidate pool* $\mathcal{A}_i$, ordered by descending

score. Thus, the ELL-SOG supports clique enumeration through column-ordered access, whereas $\mathcal{A}_i$ supports pivot selection through score-ordered access (Fig. 3, bottom right).

Compared with the original method, FastSOG avoids both the $O(N^2)$ dense SOG score-matrix storage and the later dense-row scans. Its ELL-SOG and candidate pools are generated during compaction in the exact access orders needed by enumeration and pivot selection, respectively. The scored graph therefore occupies $O(NL)$ space, and downstream stages neither reconstruct an adjacency structure nor re-sort its edges. The dense binary FOG adjacency matrix is still retained for first-order lookups.

3.2 Dataflow-Optimized 3-Clique Search

Building on the two complementary representations produced by FFSOG, the search stage follows the pivot-based 3-clique objective of TurboReg [9] to generate a bounded set of geometrically consistent correspondence triples. Given a pivot budget K and a per-pivot clique budget T , Phase 1 selects the K highest-scoring edges from the candidate union $\tilde{\mathcal{E}} = \bigcup_i \mathcal{A}_i$ as *pivots*. This candidate-restricted selection approximates the global top- K over all positive SOG edges, as evaluated in Sec. 4.4, and records each pivot as (u, v, s_{uv}) . Phase 2 intersects the column-sorted ELL-SOG neighborhoods of each pivot’s endpoints, scores their common neighbors w , and retains at most the T highest-scoring 3-cliques (u, v, w) for each pivot. We use $T=2$ in all experiments. The search therefore emits at most KT scored triples; because only the strict upper triangle is stored, every emitted triple satisfies $u < v < w$ and cannot be generated twice from different pivots.

3.2.1 Search Objective and Data-Access Mismatch

Let $\mathbf{M}$ denote the SOG score matrix supplied to the search: $\mathbf{M} = \mathbf{M}_{\text{SOG}}$ for the dense reference, while FlashReg uses the retained entries of $\tilde{\mathbf{M}}_{\text{SOG}}$ stored in ELL-SOG. Define $\mathcal{N}^+(i) = \{j > i : [\mathbf{M}]_{ij} > 0\}$ as the upper-triangular SOG neighborhood of node i and $S(i, j) = [\mathbf{M}]_{ij}$ as the weight of edge (i, j) . The search first selects the K highest-scoring positive upper-triangular edges as pivots. Formally,

$$\mathcal{P} = \underset{u < v, S(u, v) > 0}{\text{top-}K} S(u, v). \quad (7)$$

The dense reference breaks ties at the K -th score by index. For each pivot $(u, v) \in \mathcal{P}$, the admissible third vertices are their common upper-triangular neighbors:

$$\mathcal{W}(u, v) = \mathcal{N}^+(u) \cap \mathcal{N}^+(v). \quad (8)$$

Each resulting triple receives the sum of its three edge scores:

$$q(u, v, w) = S(u, v) + S(u, w) + S(v, w). \quad (9)$$

The search retains only the T highest-scoring triples for each pivot.

A conventional implementation materializes the dense $N \times N$ SOG score matrix. Pivot selection then examines $O(N^2)$ cells, while clique enumeration scans two length- N rows for each pivot and touches $O(KN)$ entries. This is wasteful because Eqs. (7)–(9) require only positive edges, their scores, and intersections between sparse neighborhoods.

The ELL-SOG exposes exactly this information. For row i , let $\mathbf{J}_i$ and $\mathbf{Q}_i$ denote its valid neighbor indices and corresponding edge scores. Then

$$\begin{aligned} \mathcal{N}^+(i) &= \{\mathbf{J}_i[t] : 1 \leq t \leq \ell_i\}, \\ S(i, \mathbf{J}_i[t]) &= \mathbf{Q}_i[t], \end{aligned} \quad (10)$$

for $1 \leq t \leq \ell_i$. Downstream kernels inspect only this valid prefix and never read the padding. Alg. 1 can therefore evaluate Eqs. (7)–(9) without scanning a dense row or reconstructing an adjacency representation. The pseudocode uses one-based array indices and zero-based thread identifiers.

3.2.2 Phase 1: Dataflow-Integrated Pivot Selection

While constructing row i , the FFSOG block extracts its m highest-scoring retained edges into $\mathcal{A}_i$ before the row leaves shared memory. Pivot selection therefore operates on at most Nm scored edges instead of rescanning the complete SOG score matrix. Because each score is an integer common-neighbor count bounded by L , every block accumulates a local histogram of its candidate scores and merges it into a device-wide histogram H . A descending scan of H finds the threshold score θ ; edges above θ are selected directly, and the remaining slots are filled from the θ -bucket. If this boundary bucket contains more tied candidates than available slots, atomic arrival order determines which of them are retained; Sec. 4.4 evaluates the resulting variation. Each selected pivot carries its score as (u, v, s_{uv}) , so Phase 2 does

Algorithm 1 Dataflow-Optimized 3-Clique Search

Require: ELL-SOG rows $(\mathbf{J}_i, \mathbf{Q}_i, \ell_i)$ and lookup S from Eq. (10)
Require: per-row top- m candidate pools $\{\mathcal{A}_i\}$; pivot budget K ; per-pivot clique budget T ; threads per block B ; per-thread register-list capacity $\kappa \geq T$
Ensure: up to T highest-scoring valid 3-cliques for each pivot

Phase 1: dataflow-integrated pivot selection

- 1: build histogram H over the integer scores $S(e)$ of all pool edges $e \in \bigcup_i \mathcal{A}_i$
- 2: $\theta \leftarrow$ the K -th largest score among $e \in \bigcup_i \mathcal{A}_i$ $\triangleright$ scan H top-down
- 3: $\mathcal{P} \leftarrow \{e : S(e) > \theta\}$; fill from $\{e : S(e) = \theta\}$ until $|\mathcal{P}| = K$; record each pivot with its pool score as (u, v, s_{uv})

Phase 2: 3-clique enumeration (one block per pivot)

- 4: **for all** pivot $(u, v, s_{uv}) \in \mathcal{P}$ **in parallel do**
- 5: **for** thread $t = 0, \dots, B-1$ **in parallel do**
- 6: $R_t \leftarrow \{(-\infty, \emptyset)\}^\kappa$ $\triangleright$ thread-local top- κ in registers
- 7: **for** $a = t + 1$ **to** ℓ_u **step** B **do**
- 8: $w \leftarrow \mathbf{J}_u[a]$
- 9: $b \leftarrow \text{BINARYSEARCH}(\mathbf{J}_v[1 : \ell_v], w)$
- 10: **if** $b = \emptyset$ **then continue** $\triangleright w \notin \mathcal{N}^+(v)$
- 11: **end if**
- 12: $s \leftarrow s_{uv} + \mathbf{Q}_u[a] + \mathbf{Q}_v[b]$ $\triangleright$ Eq. (9)
- 13: **if** $s > \min_{\text{score}} R_t$ **then** replace that minimum with (s, w)
- 14: **end if**
- 15: **end for**
- 16: **end for**
- 17: **block barrier**; write every R_t into shared-memory buffer $\mathcal{M}$
- 18: thread 0: extract up to T valid highest-scoring entries of $\mathcal{M}$ as the 3-cliques of (u, v)
- 19: **end for**

not need to look up the pivot edge again. This compact selection avoids the dense score-matrix write, its subsequent read, and a host-side synchronization.

The deployed path fuses SOG construction and histogram-based pivot selection into one cooperative kernel; an occupancy-bounded persistent grid enables grid-wide barriers between stages. On embedded devices without cooperative-launch support, the same stages execute as consecutive kernels on one stream, preserving the dataflow and output layout.

3.2.3 Phase 2: Direct 3-Clique Enumeration

For each pivot, Phase 2 enumerates valid third-vertex extensions and retains at most the T highest-scoring 3-cliques. The ELL-SOG rows are written once to compact global memory because the two endpoints of a pivot may have been produced by different FFSOG blocks. One block is assigned to each pivot (u, v) , and its B threads stride over the ℓ_u entries of $\mathbf{J}_u$, visiting every candidate w exactly once. Each thread binary-searches its candidate in the sorted row $\mathbf{J}_v[1 : \ell_v]$. Because row v stores only columns greater than v , every candidate $w \leq v$ misses, while every hit satisfies the canonical ordering $u < v < w$. A hit also returns the position b and hence the edge score $\mathbf{Q}_v[b]$, allowing the thread to evaluate Eq. (9) with three direct ELL-SOG reads. The total intersection work is $O(\ell_u \log \ell_v)$ per pivot rather than a scan over N dense columns.

Each thread retains its κ highest-scoring candidates in registers. Because $\kappa \geq T$, discarding a lower-ranked local candidate is safe: at least T candidates processed by the same thread already have no lower scores, so the discarded candidate cannot enter the pivot's block-level top- T set after merging. Local insertion requires neither atomics nor global scratch space. After enumeration, the thread-local lists are copied once to shared memory and merged, and up to T highest-scoring triples are written to global memory for the pivot.

3.2.4 Approximation and Bounded Cost

FlashReg bounds two parts of the search. First, L caps both the voter list used to compute a row of the SOG score matrix and the number of positive edges retained in that ELL-SOG row. Second, the row-local pool exposes only its m highest-scoring retained edges to global pivot selection. These are independent restrictions: avoiding truncation by L does not by itself guarantee that the top- m pools contain every global top- K pivot from the dense score matrix. FlashReg matches the dense search only if (i) $d_i \leq L$ for every row, (ii) every edge in the dense global top- K set appears in its row's top- m pool, and (iii) pivot- and clique-boundary ties are resolved identically. Otherwise, it is a capacity-bounded approximation, whose empirical effect is reported in Sec. 4.4.

The capacities also bound the sparse search resources. The ELL-SOG and candidate pools use $O(NL + Nm)$ storage, and Phase 1 examines at most Nm edges rather than all $N(N-1)/2$ cells of the dense SOG score matrix. For each of the K pivots, Phase 2 searches one row of at most L entries against another, giving $O(L \log L)$ work per pivot and $O(KL \log L)$ overall. At most KT triples proceed to pose estimation. These bounds apply to the SOG score matrix and clique search; FlashReg still constructs the dense FOG adjacency matrix with $O(N^2)$ computation and storage.

3.3 Pose Estimation, Verification, and Refinement

The downstream stages follow TurboReg [9] and are not a contribution of this work. Each retained 3-clique produces a pose hypothesis through a batched weighted-SVD (Kabsch) solver. The hypotheses are verified against the input correspondences, and the best pose is refined by iteratively reweighted SVD.

4 Experiments

4.1 Experimental Setup

We evaluate on the indoor 3DMatch [16] and 3DLoMatch [19] benchmarks and the outdoor KITTI benchmark [30], using both FPFH [14] and FCGF [18] correspondences. Let $N_{\max}$ denote the maximum number of input correspondences retained per pair, so the actual input size satisfies $N \leq N_{\max}$. We use $N_{\max}=7000$ for indoor FPFH, $N_{\max}=6000$ for indoor FCGF, and $N_{\max}=5000$ for both descriptors on KITTI. A registration is successful when its rotation and translation errors are below 15° and 30 cm indoors, or 5° and 0.6 m on KITTI. We report registration recall (RR), mean rotation error (RE) and translation error (TE) over successful pairs, and per-pair runtime for benchmark evaluation.

We compare FlashReg with PointDSC [21], VBReg [31], RANSAC [32], FSAC-IA [33], TEASER++ [5], SC2-PCR [6], MAC [7], FastMAC [28], and TurboReg [9]. Unless otherwise stated, experiments run on the RTX 5090. Embedded-platform results use the Jetson AGX Xavier (Tab. 1).

For the indoor benchmarks, we set $\tau=0.012$ m, $r=0.15$ m, and $\tau_{\text{in}}=0.1$ m; the corresponding KITTI settings are 0.072 m, 1.0 m, and 0.6 m. Unless otherwise stated, we use $L=128$, $m=32$, $K=2000$, and $T=2$. The fused SOG/pivot kernel uses 256 threads per block, while the clique kernel uses $B=64$ threads and a per-thread register capacity of $\kappa=4$.

Runtime covers the correspondence-to-pose pipeline from graph construction through pose refinement, after one warm-up and with device synchronization. Descriptor extraction, correspondence matching, and data transfers are excluded. All methods receive the same precomputed correspondences, which are initially resident in device memory for GPU methods and host memory for CPU methods. We test each baseline on the evaluation hardware using its public implementation; FlashReg and TurboReg additionally share the downstream stages described in Sec. 3.3.

4.2 Benchmark Evaluation

On the high-overlap 3DMatch benchmark (Tab. 2), FlashReg essentially matches TurboReg, reaching 84.01% RR with FPFH and 93.41% with FCGF. It also remains competitive with the strongest CPU clique method, trailing MAC by only 0.39 percentage points under FPFH and 0.24 points under FCGF. FlashReg reduces correspondence-to-pose latency from TurboReg’s 10 ms to 4–5 ms and is over two orders of magnitude faster than the CPU clique solvers. Fig. 1 highlights this accuracy–speed trade-off.

The same pattern holds on the more challenging 3DLoMatch benchmark (Tab. 3). FlashReg obtains 39.25% RR with FPFH and 59.12% with FCGF, exceeding TurboReg by 0.06 and 0.67 percentage points, respectively, while remaining within 1.74 and 0.90 points of MAC. Its latency remains 4–5 ms per pair, more than $2\times$ faster than TurboReg and about $200\times$ faster than MAC. Among successful registrations, FlashReg also reduces translation error relative to TurboReg for both descriptors.

On the outdoor KITTI benchmark (Tab. 4), FlashReg matches TurboReg’s recall exactly, with 97.83% under FPFH and 97.66% under FCGF, while outperforming all evaluated CPU methods. It reduces TurboReg’s 11–12 ms latency to 4–5 ms. Across all three benchmarks, FlashReg therefore preserves the accuracy of GPU clique-based registration while consistently delivering a $2\text{--}3\times$ speedup.

4.3 Embedded Platform Evaluation

We evaluate embedded deployment by running FlashReg and TurboReg on the Jetson AGX Xavier. On 3DMatch, we sweep $N_{\max}$ for pivot budgets $K \in \{1000, 1500, 2000\}$. The Jetson operates in its maximum-power mode with clocks locked by `nvpmode1 -m 0` and `jetson_clocks`. For every $(N_{\max}, K)$ setting, both methods process the same

Table 1: Hardware and software configurations used in the experiments.

Device	Desktop PC	Jetson AGX Xavier
CPU	AMD Ryzen 7 9850X3D	NVIDIA Carmel
GPU	NVIDIA GeForce RTX 5090	512-core NVIDIA Volta
GPU memory	32 GB (VRAM)	16 GB (unified)
CUDA version	13.0	10.2

Table 2: Performance comparison of different registration methods on the 3DMatch dataset.

Method	FPFH				FCGF			
	RR (%)	RE (deg)	TE (cm)	Time (s)	RR (%)	RE (deg)	TE (cm)	Time (s)
<i>Learning-based</i>								
PointDSC*	77.51	2.07	6.45	0.035	92.91	2.04	6.52	0.034
VBReg*	79.54	2.27	7.01	0.072	93.04	2.07	6.57	0.079
<i>Learning-free</i>								
RANSAC-10k	70.10	4.13	11.63	2.533	81.80	3.71	11.12	1.760
FSAC-IA	69.58	4.13	12.05	0.247	82.62	3.79	11.34	0.215
TEASER++	80.90	2.30	7.13	0.361	88.54	2.23	7.46	1.039
SC2-PCR	83.80	2.12	10.66	0.776	93.16	2.06	6.53	0.783
MAC	84.40	2.09	6.73	1.015	93.65	2.03	6.52	0.911
FastMAC	82.50	2.13	6.75	0.545	92.98	2.01	6.49	0.521
TurboReg*	84.17	2.04	6.57	0.010	93.59	2.03	6.46	0.010
FlashReg*	84.01	2.04	6.56	0.004	93.41	2.03	6.48	0.005

* GPU-based method; the others run on CPU. All experiments were conducted on the desktop PC (RTX 5090).

complete set of 1,623 pairs. The reported energy is the total GPU-rail energy for that run, obtained by sampling tegrastats every 20 ms and integrating the sum of rails whose reported names contain “GPU”; idle power is not subtracted. Peak allocated tensor memory is the maximum warm-up-excluded live-tensor allocation reported by the PyTorch CUDA allocator and includes both input correspondence tensors and algorithm buffers; it excludes the CUDA context, driver memory, and allocations outside the PyTorch allocator.

Fig. 4 reports recall, latency, total GPU-rail energy, and peak allocated tensor memory across these settings. FlashReg tracks TurboReg’s recall while using about half as much peak allocated tensor memory and consistently less measured energy. Eliminating the dense SOG score matrix substantially reduces tensor-memory growth over the evaluated $N_{\max}$ range, although both methods still retain the dense FOG adjacency matrix. This reduced growth is particularly valuable on the Jetson’s unified-memory architecture.

4.4 Ablation Study

Tab. 5 isolates the contribution of each FlashReg component on 3DMatch with FPFH. FastFOG and FastSOG accelerate their respective stages from 1.64 to 0.19,ms (8.6 $\times$) and from 3.56 to 0.84,ms (4.2 $\times$). Combined, they reduce the total latency from 11.40 to 7.22,ms, with only a 0.13 percentage-point change in RR. The dataflow design further reduces clique enumeration from 0.54 to 0.04,ms and reorganizes SOG construction and pivot selection; the fused implementation completes these operations in 0.42,ms. With all components enabled, the split and fused pipelines achieve total latencies of 5.16 and 5.11,ms, respectively. Overall, the deployed fused pipeline delivers a 2.23 $\times$ end-to-end speedup (55.2% latency reduction) over the dense reference while maintaining 84.01% RR, only 0.16 percentage points lower than the reference.

5 Conclusion

This work presented FlashReg, a GPU-native 3-clique correspondence-to-pose estimator that avoids materializing the dense SOG score matrix and eliminates redundant data movement between graph construction and search. Its consumer-oriented ELL-SOG and candidate pools preserve recall comparable to TurboReg while reducing estimator latency by 2–3 $\times$. On the Jetson AGX Xavier, the same design also lowers peak allocated tensor memory and measured

Table 3: Performance comparison of different registration methods on the 3DLoMatch dataset.

Method	FPFH				FCGF			
	RR (%)	RE (deg)	TE (cm)	Time (s)	RR (%)	RE (deg)	TE (cm)	Time (s)
<i>Learning-based</i>								
PointDSC*	27.51	4.24	10.42	0.032	55.14	3.85	10.45	0.033
VBReg*	33.58	4.38	10.77	0.074	57.94	3.80	10.75	0.074
<i>Learning-free</i>								
RANSAC-10k	5.86	6.21	14.13	2.257	27.24	5.08	13.62	1.770
FSAC-IA	6.26	5.91	14.62	0.235	27.27	5.03	14.01	0.201
TEASER++	34.03	4.30	10.68	0.476	48.85	4.29	11.92	0.516
SC2-PCR	36.16	4.23	10.66	0.598	57.89	3.78	10.58	0.539
MAC	40.99	4.03	10.75	0.785	60.02	3.73	10.55	1.091
FastMAC	38.24	4.13	10.82	0.461	57.61	3.71	10.50	0.603
TurboReg*	39.19	4.49	15.34	0.011	58.45	2.49	9.83	0.011
FlashReg*	39.25	4.04	10.34	0.004	59.12	2.49	7.96	0.005

* GPU-based method; the others run on CPU. All experiments were conducted on the desktop PC (RTX 5090).

Table 4: Performance comparison of different registration methods on the KITTI dataset.

Method	FPFH				FCGF			
	RR (%)	RE (deg)	TE (cm)	Time (s)	RR (%)	RE (deg)	TE (cm)	Time (s)
<i>Learning-based</i>								
PointDSC*	97.29	0.44	9.56	0.047	98.74	0.21	11.01	0.046
VBReg*	97.83	0.43	9.48	0.054	98.92	0.28	10.92	0.055
<i>Learning-free</i>								
RANSAC-10k	47.21	1.57	28.69	1.985	80.36	0.60	20.42	0.433
FSAC-IA	47.21	1.60	30.01	0.254	81.02	0.60	19.94	0.068
TEASER++	93.17	0.50	17.98	0.448	94.64	0.28	13.69	2.215
SC2-PCR	94.56	0.28	20.08	1.715	96.56	0.36	8.18	1.509
MAC	97.11	0.39	8.49	0.648	97.48	0.36	8.14	0.201
FastMAC	97.12	0.43	9.06	0.348	97.30	0.43	9.06	0.051
TurboReg*	97.83	0.41	8.13	0.011	97.66	0.36	7.72	0.012
FlashReg*	97.83	0.41	8.13	0.004	97.66	0.35	7.67	0.005

* GPU-based method; the others run on CPU. All experiments were conducted on the desktop PC (RTX 5090).

Table 5: Component ablation on 3DMatch with FPFH. Total includes verification, refinement, and pose solving. Full (fused) is the deployed pipeline. All ablation experiments were conducted on the desktop PC (RTX 5090).

Configuration	Modules enabled				RR (%)	Per-stage GPU time (ms)				
	FastFOG	FastSOG	Dataflow	Pivot		FOG	SOG	Pivot	Clique	Total
Dense reference					84.17	1.64	3.56	1.21	0.55	11.40
FastFOG	✓				84.17	0.19	3.56	1.21	0.55	9.95
FastSOG		✓			84.04	1.64	0.84	1.19	0.54	8.69
FastFOG + FastSOG	✓	✓			84.04	0.19	0.84	1.19	0.54	7.22
FastSOG + Dataflow (fused)		✓	✓	fused	84.01	1.64	0.42	–	0.04	6.55
FastSOG + Dataflow (split)		✓	✓	split	84.04	1.64	0.39	0.09	0.04	6.60
Full (split)	✓	✓	✓	split	84.04	0.19	0.39	0.09	0.04	5.16
Full (fused)	✓	✓	✓	fused	84.01	0.19	0.41	–	0.04	5.11

GPU-rail energy, making FlashReg a practical backend for onboard registration pipelines. Future work will extend the dataflow design to higher-order geometric reasoning and integrate it with complete perception systems.

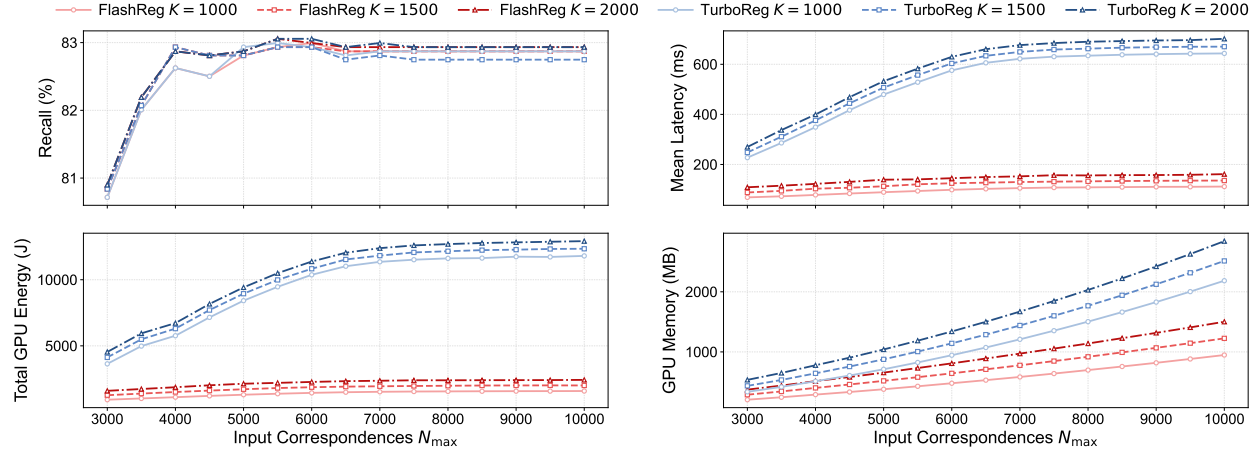


Figure 4: Recall, latency, total GPU-rail energy, and peak PyTorch-allocated tensor memory on the Jetson AGX Xavier (3DMatch) as $N_{\max}$ and K vary. Energy is integrated over the complete 1,623-pair run. The “GPU Memory” panel denotes this allocator metric, not total device-memory use; the recall axis is truncated to expose small differences.

References

- [1] François Pomerleau, Francis Colas, and Roland Siegwart. A review of point cloud registration algorithms for mobile robotics. *Foundations and Trends in Robotics*, 4(1), 2015.
- [2] Ji Zhang and Sanjiv Singh. Loam: Lidar odometry and mapping in real-time. In *Robotics: Science and Systems*, volume 2, 2014.
- [3] Ignacio Vizzo, Tiziano Guadagnino, Benedikt Mersch, Louis Wiesmann, Jens Behley, and Cyrill Stachniss. Kiss-icp: In defense of point-to-point icp – simple, accurate, and robust registration if done the right way. *IEEE Robotics and Automation Letters*, 8(2):1029–1036, 2023.
- [4] Marius Leordeanu and Martial Hebert. A spectral technique for correspondence problems using pairwise constraints. In *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, volume 2, pages 1482–1489, 2005.
- [5] Heng Yang, Jingnan Shi, and Luca Carlone. Teaser: Fast and certifiable point cloud registration. *IEEE Trans. Robot.*, 37(2):314–333, 2020.
- [6] Zhi Chen, Kun Sun, Fan Yang, and Wenbing Tao. Sc2-pcr: A second order spatial compatibility for efficient and robust point cloud registration. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 13221–13231, 2022.
- [7] Xiyu Zhang, Jiaqi Yang, Shikun Zhang, and Yanning Zhang. 3d registration with maximal cliques. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 17745–17754, 2023.
- [8] Xiyu Zhang, Yanning Zhang, and Jiaqi Yang. Mac++: Going further with maximal cliques for 3d registration. In *2025 International Conference on 3D Vision (3DV)*, pages 261–275. IEEE, 2025.
- [9] Shaocheng Yan, Pengcheng Shi, Zhenjun Zhao, Kaixin Wang, Kuang Cao, Ji Wu, and Jiayuan Li. Turboreg: Turboclique for robust and efficient point cloud registration. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, pages 26371–26381, 2025.
- [10] Paul J Besl and Neil D McKay. A method for registration of 3-d shapes. *IEEE Trans. Pattern Anal. Mach. Intell.*, 14(2):239–256, 1992.
- [11] Jiaolong Yang, Hongdong Li, Dylan Campbell, and Yunde Jia. Go-icp: A globally optimal solution to 3d icp point-set registration. *IEEE Trans. Pattern Anal. Mach. Intell.*, 38(11):2241–2254, 2016.
- [12] Andrew E Johnson and Martial Hebert. Using spin images for efficient object recognition in cluttered 3d scenes. *IEEE Trans. Pattern Anal. Mach. Intell.*, 21(5):433–449, 1999.
- [13] Radu Bogdan Rusu, Nico Blodow, Zoltan Csaba Marton, and Michael Beetz. Aligning point cloud views using persistent feature histograms. In *2008 Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pages 3384–3391, 2008.

- [14] Radu Bogdan Rusu, Nico Blodow, and Michael Beetz. Fast point feature histograms (fpfh) for 3d registration. In *2009 Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pages 3212–3217, 2009.
- [15] Federico Tombari, Samuele Salti, and Luigi Di Stefano. Unique signatures of histograms for local surface description. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*, pages 356–369, 2010.
- [16] Andy Zeng, Shuran Song, Matthias Nießner, Matthew Fisher, Jianxiong Xiao, and Thomas Funkhouser. 3dmatch: Learning local geometric descriptors from rgb-d reconstructions. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 1802–1811, 2017.
- [17] Zan Gojcic, Caifa Zhou, Jan D Wegner, and Andreas Wieser. The perfect match: 3d point cloud matching with smoothed densities. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019.
- [18] Christopher Choy, Jaesik Park, and Vladlen Koltun. Fully convolutional geometric features. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, pages 8958–8966, 2019.
- [19] Shengyu Huang, Zan Gojcic, Mikhail Usvyatsov, Andreas Wieser, and Konrad Schindler. Predator: Registration of 3d point clouds with low overlap. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 4267–4276, 2021.
- [20] Zheng Qin, Hao Yu, Changjian Wang, Yulan Guo, Yuxing Peng, and Kai Xu. Geometric transformer for fast and robust point cloud registration. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 11143–11152, 2022.
- [21] Xuyang Bai, Zixin Luo, Lei Zhou, Hongkai Chen, Lei Li, Zeyu Hu, Hongbo Fu, and Chiew-Lan Tai. Pointdsc: Robust point cloud registration using deep spatial consistency. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 15859–15869, 2021.
- [22] Christopher Choy, Wei Dong, and Vladlen Koltun. Deep global registration. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 2514–2523, 2020.
- [23] G Dias Pais, Srikumar Ramalingam, Venu Madhav Govindu, Jacinto C Nascimento, Rama Chellappa, and Pedro Miraldo. 3dregnet: A deep neural network for 3d point registration. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 7193–7203, 2020.
- [24] Kenji Koide, Masashi Yokozuka, Shuji Oishi, and Atsuhiko Banno. Voxelized gicp for fast and accurate 3d point cloud registration. In *2021 Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pages 11054–11059, 2021.
- [25] Weimin Wang and Qiong Chang. Van-icp: Gpu-accelerated approximate nearest neighbor search for icp registration via voxel dilation. In *ICASSP 2023-2023 IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, pages 1–5, 2023.
- [26] Qiong Chang, Weimin Wang, and Jun Miyazaki. Accelerating nearest neighbor search in 3d point cloud registration on gpus. *ACM Trans. Archit. Code Optim.*, 22(1):1–24, 2025.
- [27] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Fast global registration. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*, pages 766–782, 2016.
- [28] Yifei Zhang, Hao Zhao, Hongyang Li, and Siheng Chen. Fastmac: Stochastic spectral sampling of correspondence graph. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 17857–17867, 2024.
- [29] Francisco Vázquez, José Jesús Fernández, and Ester M. Garzón. A new approach for sparse matrix vector product on NVIDIA GPUs. *Concurr. Comput. Pract. Exp.*, 23(8):815–826, 2011.
- [30] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 3354–3361, 2012.
- [31] Haobo Jiang, Zheng Dang, Zhen Wei, Jin Xie, Jian Yang, and Mathieu Salzmann. Robust outlier rejection for 3d registration with variational bayes. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 1148–1157, 2023.
- [32] Martin A Fischler and Robert C Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Commun. ACM*, 24(6):381–395, 1981.
- [33] Ziyang Yu, Qiong Chang, and Jun Miyazaki. Fsac-ia: A hierarchical constructed sac-ia algorithm for point cloud alignment acceleration. In *Proc. IEEE Int. Conf. Image Process. (ICIP)*, pages 1486–1491, 2025.